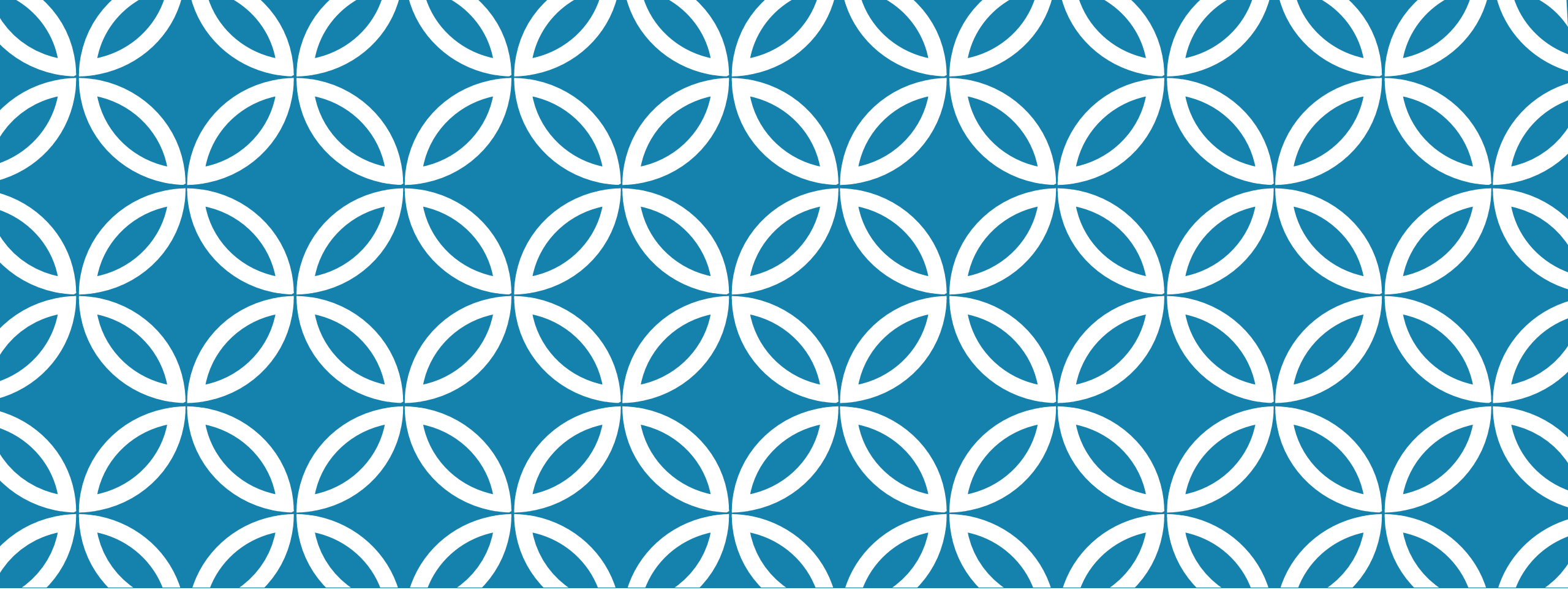


NESNE TABANLI PROGRAMLAMA-I

Öğr. Gör. Erhan AKAGÜNDÜZ

SOYUTLAMA NEDİR?

- Soyutlama, karmaşık sistemleri basitleştirmek için kullanılan güçlü bir OOP tekniğidir.
- Bir nesnenin sadece gerekli olan kısımlarını gösterip, detayları gizlememize olanak sağlar.
- Kullanıcıya sadece neyin yapılacağını gösterir, nasıl yapılacağı detayını saklar.
- Mesela bir arabayı çalıştırmak için marşa basarsınız, ama motorun içinde neler olduğunu bilmenize gerek yoktur.
- Bu sayede kullanıcı gereksiz detaylardan korunur ve sistem daha anlaşılır hale gelir. Python'da soyutlamayı "abstract class" ve "abstract method" kullanarak yaparız.

NEDEN SOYUTLAMA KULLANIRIZ?

- Soyutlamanın en önemli amacı karmaşıklığı gizlemek ve standart bir yapı sağlamaktır.
- Gerçek hayatta çoğu nesnenin ayrıntılarına erişmeyiz, sadece belirli işlevlerini kullanırız.
- Yazılım dünyasında da soyutlama, kodun okunabilirliğini ve bakımını kolaylaştırır.
- Ayrıca ekip çalışmasında, her geliştiricinin sadece gerekli kısımlara odaklanmasına imkan tanır.
- Böylece iş bölümü yapılırken kimse gereksiz detaylarla uğraşmaz.
- Sonuç olarak daha düzenli, daha modüler ve genişletilebilir projeler ortaya çıkar.

PYTHON'DA SOYUTLAMA NASIL YAPILIR?

- Python'da soyutlama için **abc (Abstract Base Class)** modülünü kullanırız.
- Bu modül, bir sınıfın "soyut sınıf" olarak tanımlanmasını sağlar.
- Soyut sınıflar doğrudan nesne üretmek için değil, miras alınarak kullanılmak için yazılır.
- Bir soyut sınıfta tanımlanan "abstract method", mutlaka alt sınıflar tarafından uygulanmalıdır.
- Aksi halde hata alırız ve sınıfımız eksik kalır.
- Şimdi küçük bir örnekle gösterelim.

BASİT SOYUT SINIF ÖRNEĞİ

- Bu örnekte **Animal** sınıfı bir soyut sınıftır.
- **sound** metodu soyut olarak tanımlandı, yani gövdesizdir.
- Bu sınıftan doğrudan nesne üretemeyiz.
- Çünkü soyut sınıflar sadece rehberlik etmek için vardır.
- Alt sınıflar, bu metodun içini doldurmak zorundadır.
- Şimdi alt sınıflarla devam edelim.

```python
from abc import ABC, abstractmethod

class Animal(ABC):
    @abstractmethod
    def sound(self):
        pass
```

SOYUTLAMA İLE ALT SINIF KULLANIMI

- Burada Dog ve Cat sınıfları, Animal soyut sınıfını miras aldı.
- Her biri sound metodunu kendine göre tanımlamak zorunda kaldı.
- Eğer tanımlamasaydık, Python hata fırlatacaktı.
- Çıktıya baktığımızda:

```
Hav Hav
Miyav
```

```python
class Dog(Animal):
    def sound(self):
        return "Hav Hav"

class Cat(Animal):
    def sound(self):
        return "Miyav"

dog = Dog()
cat = Cat()
print(dog.sound())
print(cat.sound())
```

SOYUTLAMA İLE STANDART BELİRLEME

- Soyutlama bize projelerde standart bir çatı kurma imkanı sağlar.
- Örneğin tüm hayvan sınıflarının mutlaka bir sound metoduna sahip olmasını garanti ettik.
- Bu sayede farklı sınıflardan nesneler üretsek bile, hepsini aynı **metodla** kullanabiliyoruz.
- Kullanıcı, hayvanın kedi mi köpek mi olduğunu bilmek zorunda kalmaz.
- Sadece **sound( )** metodunu çağırır ve sonucu alır.
  Böylece sistem güvenli ve düzenli çalışır.

SOYUT SINIFLARDAN NESNE ÜRETİLEMEZ

- Soyut sınıfların doğrudan nesnesi oluşturulamaz.
- Çünkü içinde tamamlanmamış (abstract) metotlar vardır.
- Eğer aşağıdaki gibi deneme yaparsak:

```
a = Animal()
```

- Python bize TypeError: Can't instantiate abstract class Animal hatasını verir.
- Bu kural sayesinde, soyut sınıf sadece rehber niteliğinde kalır.
- Gerçek kullanım her zaman alt sınıflar üzerinden yapılır.

GERÇEK HAYATTAN ÖRNEK

- Bir üniversite sistemi geliştirdiğimizi düşünelim.
- Tüm fakültelerin öğrencileri var, ama her fakültenin öğrenci kayıt şekli farklı olabilir.
- Bu durumda **Student** isminde soyut bir sınıf tanımlarız.
- İçine **register( )** isimli soyut bir metod koyarız.
- Alt sınıflar olan **EngineeringStudent**, **MedicalStudent** bu metodun içini kendine göre doldurur.
- Böylece tüm fakülteler aynı standarda uymak zorunda kalır.

KODLA ÜNİVERSİTE ÖRNEĞİ

```
class Student(ABC):
    @abstractmethod
    def register(self):
        pass

class EngineeringStudent(Student):
    def register(self):
        return "Mühendislik Fakültesine kayıt yapıldı."

class MedicalStudent(Student):
    def register(self):
        return "Tıp Fakültesine kayıt yapıldı."

s1 = EngineeringStudent()
s2 = MedicalStudent()
print(s1.register())
print(s2.register())
```

Çıktı:

```
Mühendislik Fakültesine kayıt yapıldı.
Tıp Fakültesine kayıt yapıldı.
```

SOYUTLAMA VE ENCAPSULATİON İLİŞKİSİ

- Soyutlama ile encapsulation çoğu zaman karıştırılır.
- Encapsulation verilerin gizlenmesini sağlarken, soyutlama gereksiz detayların saklanmasını sağlar.
- Encapsulation → “veri güvenliği”, Abstraction → “gereksiz ayrıntıyı gizleme” diyebiliriz.
- Örneğin sınav notunu private yapmak encapsulation’dır.
- Ama öğrenci kaydını farklı fakültelere göre standartlaştırmak abstraction’dır.
- İkisi birlikte kullanıldığında çok güçlü yazılım mimarileri ortaya çıkar.

ABSTRACTİON KULLANMAZSAK NE OLUR?

- Eğer soyutlama yapmazsak, her sınıf kendi metodlarını farklı isimlerle tanımlayabilir.
- Örneğin bir sınıfta **kayitOl( )**, diğerinde **ogrenciEkle( )** gibi farklı isimler olur.
- Bu durum, kodu kullanan kişiler için ciddi kafa karışıklığı yaratır.
- Ayrıca kodun bakım maliyeti artar.
- Soyutlama, tüm sınıfları aynı çatı altında toplar.
- Böylece sistem daha öngörülebilir hale gelir.

SOYUTLAMA İLE POLYMORPHİSM İLİŞKİSİ

- Soyutlama, polymorphism için bir altyapı hazırlar.
- Soyut sınıfta tanımlanan metod, alt sınıflarda farklı biçimlerde uygulanabilir.
- Örneğin sound() metodunu kedi, köpek, kuş farklı seslerle doldurabilir.
- Bu, aslında polymorphism'in temelidir.
- Yani soyutlama olmadan çok biçimlilik eksik kalır.
- O yüzden bu konular OOP'nin birbirini tamamlayan parçalarıdır.

ABSTRACTİON AVANTAJLARI

- Kodun okunabilirliği artar.
- Ekip çalışmasında ortak standart sağlar.
- Gereksiz detaylar saklanarak sadelik oluşturur.
- Daha güvenli ve bakımı kolay projeler çıkar.
- Polymorphism gibi diğer OOP özelliklerinin temelini hazırlar.
- Büyük projelerde hata payını düşürür.

ABSTRACTİON DEZAVANTAJLARI

- Her güzel özelliğin bazı zorlukları da vardır.
- Soyutlama, bazen fazla standart koyduğu için geliştiricilerin esnekliğini kısıtlayabilir.
- Ayrıca küçük projelerde soyutlama yapmak gereksiz olabilir.
- Soyut sınıfları anlamak, yeni başlayanlar için biraz zaman alabilir.
- Yanlış kullanıldığında gereksiz karmaşıklık yaratabilir.
- Bu nedenle abstraction özellikle büyük ve çok geliştiricili projelerde tercih edilmelidir.

DERS ÖZETİ

- Soyutlama, yazılım dünyasında karmaşayı önleyen en önemli yöntemlerden biridir.
- Python’da soyutlama abc modülü ile yapılır.
- Soyut sınıflardan nesne üretilemez, ama alt sınıflar üzerinden kullanılır.
- Gerçek hayattaki “detayları saklama, önemli olanı gösterme” prensibini yazılıma taşır.
- Encapsulation, Inheritance ve Polymorphism ile birlikte OOP’nin temel direğidir.
- Bir sonraki dersimizde, soyutlamayla bağlantılı Polymorphism (Çok Biçimlilik) konusuna geçeceğiz.